Compiler Construction Lab Assignment 1

Ria Vinod, 23070122179, Batch B1, Date: 13th July

Compiler Construction Lab Assignment - 1
Ria Vinod, 23070122179, Batch B1

LEX (Lexical Analyser)

LEX stands for Lexical Analyser Generator. It is a UNIX utility used to generate lexical analysers (scanners). A lexical analyser processes the given source program one character at a time, groups the characters into meaningful units called tokens & forwards these tokens to the parser.

A lex specification consists of regular expressions & the actions associated with them. From this specification, LEX automatically generates a C source file (lex.yy.c) that performs lexical analysis.

ROLE OF LEX

→ LEX performs the first phase of a compiler, known as lexical analysis.

→ its primary functions are
- reading source code character by character
- identifying valid lexemes & converting them into tokens.
- ignoring unnecessary spaces, tabs & comments
- detecting lexical errors
- sending the generated tokens to the parser (YACC).

→ example: int a = 10;
output tokens:
- KEYWORD (int)
- NUMBER (10)
- IDENTIFIER (a)
- SEMICOLON (;)
- ASSIGN (=)

↳ this is also known as token recognition

BLOCK SCHEMATIC OF LEX

source program
↓
lex program (Regex + Rules)
↓
Lex Utility
↓
Generated file (lex.yy.c)
↓
C compiler (gcc)
↓
Executable file
↓
source code input
↓
sequence of token
↓
parser (YACC)

FILE FORMAT OF LEX

→ A file is divided into 3 sections, separated by %%.

→ Definitions
%%
Rules
%%
User Subroutines

→ example

```
%{
#include<stdio.h>
%}
digit [0-9]
%%
{digit}+    {printf("Number\n");}
%%
int main()
{
    yylex();
    return 0;
}
```

SECTIONS OF A LEX PROGRAM

→ Definition section
- contains files like header files, variable declarations, macro definitions, regular definitions & start state definitions declarations.

• Eg:
```
%{
#include<stdio.h>
%}
digit [0-9]
letter [a-zA-Z]
```

→ Rules Section
- This section contains patterns & their corresponding actions
- eg:

pattern	action
[0-9]+	{printf("Number");}

→ user subroutines section

- this section contains user defined C functions such as main(), helper functions, yywrap().
- Example:
```
int main()
{
 yylex();
 return 0;
}
int yywrap;
{
 return 1;
}
```

IMPORTANT FUNCTIONS IN LEX

→ yylex() : main scanner function generated by LEX. reads input, matches patterns & executes actions
- eg: yylex();

→ yywrap() : called when EOF is reached
- return 1 ; → stop scanning
- return 0 ; → continue scanning
- eg:
```
int yywrap()
 {return 1;
 }
```

→ input() - reads one char from input
- ~~reads~~ eg: char c = input();

→ unput() - pushes a char back into input stream
- eg - unput(c);

→ ECHO : prints matched input exactly as it is
- eg: ECHO;

IMPORTANT VARIABLES IN LEX

→ yytext - stores current matched lexeme
- eg: printf("%s", yytext);

→ yyleng - stores length of matched lexeme
- eg: printf("%d", yyleng);

→ yyin - input file pointer
- eg: yyin = fopen("input.txt","r");

→ yyout - output file pointer
- eg: yyout = fopen("output.txt","w");

→ yylineno - stores current line number
- eg: printf("%d", yylineno);

STEPS TO COMPILE A LEX PROGRAM

1) Generate C source file
   lex sample.l creates lex.yy.c
2) Compile generated C program
   gcc lex.yy.c -o sample -ll or gcc lex.yy.c -ll
3) Execute the program
   - ./sample
4) Enter input
   Hello
   123
5) Observe generated output

TOKEN SPECIFICATION

TOKEN	regex
Identifier	[a-zA-Z_][a-zA-Z0-9_]*
Integer	[0-9]+
float	[0-9]+\.[0-9]+
keyword	"if"
operator	[+\-*/=]
Whitespace	[\t\n]+

SAMPLE PROGRAM

```
%{
  #include <stdio.h>
%}
%%
  [a-zA-Z]+ {printf("word\n"); }
%%

int main()
{
  yylex();
  return 0;
}
int yywrap()
{
  return 1;
}
```

Input : Hello
Output : Word.

APPLICATIONS OF LEX

- compiler construction
- syntax highlighting in code editors
- Text & pattern processing.
- lexical analysis (token generation)
- source code preprocessing

ADVANTAGES

→ Automatically generates lexical analysers
→ uses regex for easy token specification
→ integrates easily with YACC for compiler dvpt.

DISADVANTAGES

→ performs only lexical analysis
→ can't detect syntax or semantic errors
→ difficult to handle context sensitive patterns.

YACC (YET ANOTHER COMPILER COMPILER)

→ YACC is a parser generator used to create the syntax analyser of a compiler. It accepts grammar rules (CFG/ BNF) with semantic actions & generates a C source file y.tab.c.
→ It works with LEX, which supplies the tokens for syntax analysis

Role of YACC

→ receiving tokens from LEX
→ checking grammar rules
→ Building parse tree
→ reporting syntax errors
→ executing semantic actions.

BLOCK DIAGRAM

source program
↓
LEX (scanner)
↓
TOKENS
↓
YACC (Parser)
↓
y.tab.c
↓
gcc
↓
Executable Parser
↓
Parse Tree / Syntax Check

YACC FILE FORMAT

→ YACC program has 3 sections separated by %%.

→ Definitions
%%
GrammarRules
%%
User Functions

SECTIONS OF YACC

→ Definition section –

- contains header files, variables, %token, %start, operator precedence
- eg:
```
%{
#include <stdio.h>
%}
%token NUM ID
```

Date

→ Grammar Rules section - contains grammar productions & actions

```
expr : expr '+' expr
     | NUM;
%%
```

→ User functions section

- contains main(), yyparse(), yyerror(), helper functions

IMPORTANT FUNCTIONS

→ yyparse() – starts parsing & checks grammar

- eg: yyparse();

→ yyerror() – handles syntax errors

- eg: yyerror(){ printf("Syntax error"); }

→ yylex() – gets the next token from LEX.

- eg: yylex();

IMPORTANT VARIABLES

→ yyval : stores token value

- eg: yylval = 10;

→ yychar : current look-ahead token

- eg: printf("%d", yychar);

→ yydebug : enables parser debugger

- eg: yydebug = 1;

→ yynerrs : stores syntax error count

- eg: Printf("%d", yynerrs);

STEPS TO COMPILE A YACC PROGRAM

→ suppose the files are
sample.y sample.l

1) Generate the parser
yacc -d sample.y
this generates
y.tab.c y.tab.h

2) Generate the lexical analyzer
lex sample.l
this generates lex.yy.c

3) Compile both files
gcc y.tab.c lex.yy.c -o sample -ll

4) Execute the program
./sample

5) Enter input & observe the output

SPECIFICATION OF GRAMMAR

→ YACC specifies syntax of a programming lang using CFG. Each grammar rule consists of a production & an optional semantic action.

→ eg: Expression → expression + term
expression → term
term → NUMBER

Date

→ the semantic actions enclosed within {} are executed whenever corresponding rule is matched.

GRAMMAR RECOGNITION

YACC checks whether the tokens from LEX satisfy the grammar.
eg: input a+b, output: Accepted.
If not, yyerror() reports a syntax error.

SAMPLE YACC PROGRAM

```
%{
#include <stdio.h>
%}
%token NUM
%%
expr : NUM ;
%%
int main(){ yyparse(); }
int yyerror(){ printf("Syntax Error"); }
```

Input : 5 , output : Accepted

APPLICATION

→ Compiler Construction → Parser Generation
→ Expression Evaluation → Syntax Analysis
→ Code translation

ADVANTAGES

→ Automatically generates parsers
→ Easy integration with LEX.
→ Detects syntax errors easily

LIMITATIONS

→ performs only syntax analysis
→ requires LEX for token generation
→ may produce shift/reduce conflicts.

Post lab Questions

Q1. What are the specifications for Regular Expressions?

Regex are the patterns used in LEX to recognise tokens in the input program. Each regular expression is associated with an action that is executed when the pattern matches.

Regex	Recognises
[A-Za-z_][A-Za-z0-9_]*	identifier
[0-9]+	integer
"if"	else
[+\-*/=]	Operator

Date

Q2. Explain with an example the sections of LEX & YACC program.

A LEX program has 3 sections separated by %%.

→ Definition section - contains header files, vars, and regex definitions.
→ Rules section - contains pattern - action pairs
→ User Functions section - contains main(), yywrap(), and helper functions

Example

```
%{
#include <stdio.h>
%}
%%
[0-9]+ printf("Number");
%%
int main()
{
  yylex();
}
```

YACC program also has 3 sections separated by %%.

→ Definition section - contains token declarations & header files
→ Grammar rules section - contains grammar productions & actions
→ User functions section - contains main(), yyparse(), yyerror(), etc.

→ eg: %% token NUM
%%
expr : NUM ;
%%
int main()
{
  yyparse();
}

Q3. With an example explain Token.

→ A token is the smallest meaningful unit of a program recognised by lexical analyser (LEX). Tokens are passed to YACC for syntax analysis.

→ Input: int sum = 10;

→

Lexemme	Token
int	Keyword
sum	Identifier
=	Operator
10	Number
;	Separator.

Q4. List commands for LEX & YACC program execution.

For lex lex filename.l
gcc lex.yy.c -o output -ll
./output.

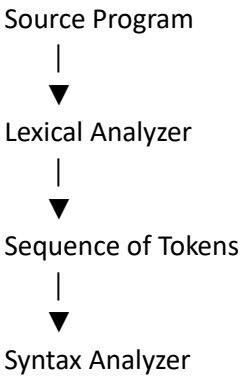
Date

For LEX & YACC together
yacc -d filename.y
lex filename.l
gcc y.tab.c lex.yy.c -o output -ll
./output.

— x —

Extra Content (Usha Ma'am's additional points)

1. Lexical Analysis Block Diagram

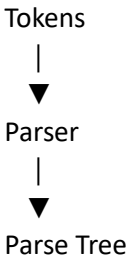


Explanation

- The source program written by the programmer is given as input to the lexical analyzer.
- The lexical analyzer (scanner) reads the program character by character from left to right.
- It groups characters into meaningful units called lexemes.
- Each lexeme is converted into a token such as a keyword, identifier, operator, constant, or separator.
- Whitespaces, tabs, newlines, and comments are ignored during scanning.
- If an invalid character is encountered, a lexical error is reported.
- The generated sequence of tokens is passed to the syntax analyzer (parser) for further processing.

---

2. Syntax Analysis Block Diagram



Explanation

- The parser receives the tokens generated by the lexical analyzer.
- It checks whether the sequence of tokens follows the grammar rules of the programming language.
- The parser uses Context-Free Grammar (CFG) to validate the syntax.
- If the token sequence is valid, it constructs a parse tree (syntax tree).
- The parse tree represents the hierarchical structure of the program.
- If the input violates grammar rules, the parser reports a syntax error using yyerror().

---

3. Structure of a LEX File

```
-------------------------
Definitions
-------------------------
%%
-------------------------
Rules
-------------------------
%%
-------------------------
User Functions
-------------------------
```

Explanation

- A LEX program is stored in a .l file.
- It consists of three sections separated by %%.
- The Definition Section contains header files, global variables, macro definitions, and regular expression definitions.
- The Rules Section contains pattern–action pairs. When a pattern matches the input, the corresponding action is executed.
- The User Functions Section contains functions such as main(), yywrap(), and any user-defined helper functions.
- This structure helps organize the lexical analyzer clearly and efficiently.

---

4. Structure of a YACC File

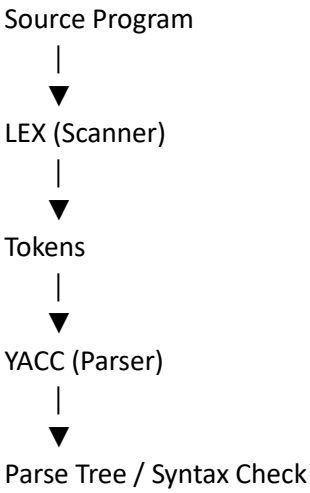
```
-------------------------
Definitions
-------------------------
%%
-------------------------
Grammar Rules
-------------------------
%%
-------------------------
User Functions
-------------------------
```

Explanation

- A YACC program is stored in a .y file.
- It also consists of three sections separated by %%.
- The Definition Section contains header files, token declarations (%token), variable declarations, precedence declarations, and the start symbol.
- The Grammar Rules Section contains grammar productions along with semantic actions written in C.
- The User Functions Section contains functions such as main(), yyparse(), yyerror(), and other user-defined functions.
- This structure allows YACC to generate an efficient parser from the grammar specification.

---

5. Working of LEX and YACC

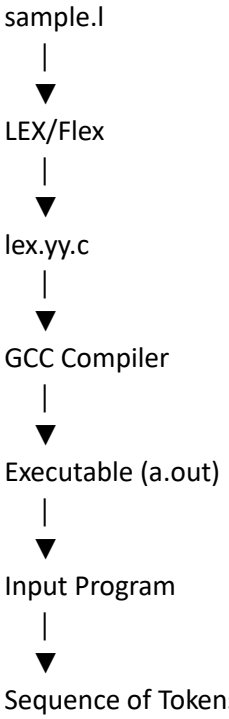


Explanation

- The source program is first given to LEX.
- LEX performs lexical analysis by reading the source code and converting it into tokens.
- These tokens represent keywords, identifiers, operators, constants, and separators.
- The generated tokens are passed to YACC.
- YACC performs syntax analysis by checking whether the tokens satisfy the grammar rules.
- If the grammar is valid, YACC builds a parse tree.
- If any syntax error is found, the parser reports it using yyerror().

---

6. Creation of a Lexical Analyzer (LEX Compilation Process)



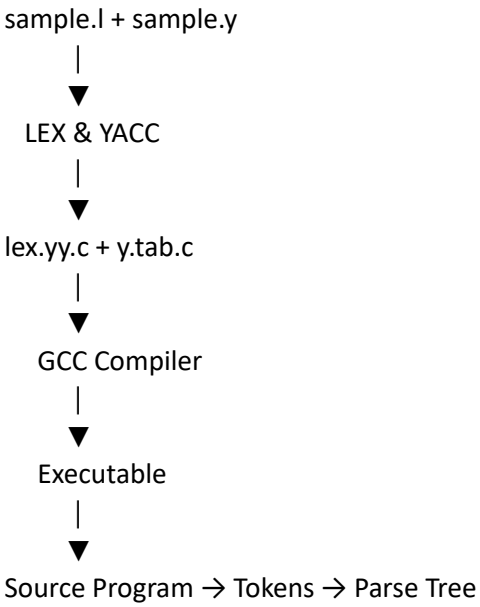
Explanation

- The programmer writes a LEX program in a file with the .l extension.

- The LEX/Flex tool processes this file and generates a C source file named lex.yy.c.
- The generated C program contains the implementation of the lexical analyzer.
- The GCC compiler compiles lex.yy.c and creates an executable program.
- When the executable is run, it reads the input program.
- The scanner recognizes lexemes using regular expressions.
- Finally, it converts the lexemes into tokens, which are then passed to the parser.

---

7. Compilation Process of LEX & YACC Together



Explanation

- The LEX file (.l) is processed by Flex to generate lex.yy.c.
- The YACC file (.y) is processed by YACC to generate y.tab.c.
- Both generated C files are compiled together using GCC.
- This produces a single executable containing both the scanner and the parser.
- When executed, the scanner first converts the source code into tokens.
- The parser then checks these tokens against the grammar and generates a parse tree or reports syntax errors.

-x-